Module 06 Lab - Regression and Classification Models

Objective: To understand the difference between regression and classification and to build your first linear models for both tasks.

In this lab, you will write the code to train and evaluate the models.

Part 1: Regression vs. Classification

This is the most fundamental distinction between types of supervised learning problems.

- **Regression:** The goal is to predict a **continuous numerical value**.
  - *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.
- **Classification:** The goal is to predict a **discrete category or class label**.
  - *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.

In this lab, we will tackle one of each.

Part 2: Linear Regression

Concept: Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".

- **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/mas

# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)
```

```
# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg
```

Task 1: Train and Evaluate a Linear Regression Model

Your Task:

1. Create an instance of the `LinearRegression` model.
2. Train the model using the training data (`X_train_reg`, `y_train_reg`).
3. Make predictions on the test data.
4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# --- ENTER YOUR CODE HERE ---
import numpy as np

# 1. Create the model instance
# lr_model = ...
lr_model = LinearRegression()

# 2. Train the model
# lr_model.fit(...)
lr_model.fit(X_train_reg, y_train_reg)

# 3. Make predictions
# y_pred_reg = lr_model.predict(...)
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
# mse = mean_squared_error(y_test_reg, y_pred_reg)
# print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
# print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is of
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is off
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning our model is off by about $58 on averag
```

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.

- **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls
```

Task 2: Train and Evaluate a Logistic Regression Model

Your Task:

1. Create an instance of the `LogisticRegression` model.
2. Train the model using the classification training data.
3. Make predictions on the test data.
4. Evaluate the model using `accuracy_score`.

```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
# log_model = ...
log_model = LogisticRegression()

# 2. Train the model
# log_model.fit(...)
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
```

```
# y_pred_cls = log_model.predict(...)
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model
# accuracy = accuracy_score(y_test_cls, y_pred_cls)
# print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

```
print(lr_model.coef_)
```

```
[ -0.43695889 -38.29728935]
```

Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?**
2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**
3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?

Q1 Answer: The key difference between regression problems and classification problems is the type of answer that the model provides for each type of problem. Regression gives you a number on a scale, so a guess could either be close to the true value or far from it. Classification, on the other hand, picks one label from a fixed list, so a guess here is either right or wrong. For example, in this lab, Fare is a regression problem because 40 dollars is between 30 and 50 dollars, so a guess that's just a few dollars wrong is still close. Survived is a classification problem despite it only showing as a 0 or 1, since there are no values between those numbers, and a passenger either survived or did not.

Q2 Answer: The output that printed was [-0.43695889 -38.29728935], which are the slopes that the model learned for each of the features in the order Age and Pclass. Each slope shows how much the predicted Fare changes when that feature goes up by one while the other feature stays the same. For example, the Age coefficient of -0.44 means that the predicted fare drops by about 0.44 dollars for each extra year of age. On the other hand, the Pclass coefficient of -38.30 means that the predicted fare drops by about 38.30 dollars each time Pclass goes up by one, such as from 1st class to 2nd class, or

from 2nd class to 3rd. Both of the numbers are negative, so the predicted fare goes down when either feature goes up. The Pclass number is also much larger in size than the Age number, which tells me that the class has a much stronger effect on fare than age does.

Q3 Answer: For the regression model, we used 'mean_squared_error' for evaluation because Fare is a continuous number, so we have to measure how far each prediction is from the real value. MSE squares each error so that the positive and negative errors don't cancel each other out. That ends up giving a heavier penalty to larger errors than to smaller ones. The MSE printed was 3364.92, and its square root is 58.01, meaning the model is off by about 58 dollars on average. Being off by 58 dollars is a large miss since many fares are much smaller than that. The slides for Module 6 also say that outliers pull the line. Back in Module 4, the pptx slides show Fare has a long tail of high values. So with only Age and Pclass features, the model can't explain those high fares, which shows that the model itself is weak.

On the other hand, we used accuracy_score for the classification model because each prediction is either the correct label or the wrong label, and the model got 74.83% of the test passengers right. Accuracy is not a good metric for the fare prediction task because it only counts the exact matches. For example, a prediction of 45.37 dollars for a real fare of 45.50 dollars might be close, but accuracy would count it as wrong. Almost every fare prediction misses by at least a few cents, so accuracy would end up being close to 0%, which means that it doesn't show how close the model's predictions actually are.